

VI. CONCLUSION

This paper highlights Chain-of-Thought (CoT) hijacking as a concrete safety challenge in open-weight LLM ecosystems. The core risk is not only malicious final outputs, but also manipulated observable reasoning that can appear trustworthy to users while being trigger-controlled. This makes CoT a security-relevant surface, especially when lightweight adapters can be distributed and attached at scale.

To study this threat, we propose Multiple Reverse Tree Search (MRTS) for synthesizing output-aligned malicious CoTs from prompt–output pairs, and Two-stage Backdoor Hijacking (TSBH) for separating trigger-conditioned output controllability from CoT-pattern imprinting during fine-tuning. Across multiple model families and settings, our results show persistent trigger-activated CoT hijacking with measurable behavioral separation between trigger-present and trigger-absent conditions. These findings motivate CoT-level auditing and evaluation protocols that jointly measure output control, CoT control, and downstream utility. Mitigation remains important, but in this work it is treated as a secondary analysis to understand utility trade-offs after hijacking.

REFERENCES

- [1] C. Lu, X. Fan, Y. Huang, R. Xu, J. Li, and W. Xu, “Does chain-of-thought reasoning really reduce harmfulness from jailbreaking?” in *ACL Findings*, 2025, pp. 6523–6546.
- [2] J. Chua, J. Betley, M. Taylor, and O. Evans, “Thought crime: Backdoors and emergent misalignment in reasoning models,” *CoRR*, vol. abs/2506.13206, 2025.
- [3] M. Kuo, J. Zhang, A. Ding, Q. Wang, L. DiValentin, Y. Bao, W. Wei, H. Li, and Y. Chen, “H-cot: Hijacking the chain-of-thought safety reasoning mechanism to jailbreak large reasoning models, including openai o1/o3, deepseek-r1, and gemini 2.0 flash thinking,” *CoRR*, vol. abs/2502.12893, 2025.
- [4] J. Zhao, T. Fu, R. Schaeffer, M. Sharma, and F. Barez, “Chain-of-thought hijacking,” *CoRR*, vol. abs/2510.26418, 2025.
- [5] T. Dong, M. Xue, G. Chen, R. Holland, Y. Meng, S. Li, Z. Liu, and H. Zhu, “The philosopher’s stone: Trojaning plugins of large language models,” in *NDSS*, 2025.
- [6] S. Liu, R. Li, L. Yu, L. Zhang, Z. Liu, and G. Jin, “Badthink: Triggered overthinking attacks on chain-of-thought reasoning in large language models,” in *AAAI*. AAAI Press, 2026, pp. 32 141–32 149.
- [7] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *NeurIPS*, 2022.
- [8] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” in *NeurIPS*, 2022.
- [9] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, “Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting,” in *NeurIPS*, 2023.
- [10] Q. Lyu, S. Havaladar, A. Stein, L. Zhang, D. Rao, E. Wong, M. Apidanki, and C. Callison-Burch, “Faithful chain-of-thought reasoning,” in *IJCNLP-AACL, Nusa Dua, Bali, Nov. 2023*, pp. 305–329.
- [11] D. Paul, R. West, A. Bosselut, and B. Faltings, “Making reasoning matter: Measuring and improving faithfulness of chain-of-thought reasoning,” in *EMNLP Findings*, 2024, pp. 15 012–15 032.
- [12] M. Tutek, F. H. Chaleshtori, A. Marasovic, and Y. Belinkov, “Measuring chain of thought faithfulness by unlearning reasoning steps,” in *EMNLP*, 2025, pp. 9935–9960.
- [13] S. Lewis-Lim, X. Tan, Z. Zhao, and N. Aletras, “Analysing chain of thought dynamics: Active guidance or unfaithful post-hoc rationalisation?” in *EMNLP*, 2025, pp. 29 838–29 853.
- [14] Y. Chen, J. Benton, A. Radhakrishnan, J. Uesato, C. Denison, J. Schulman, A. Somani, P. Hase, M. Wagner, F. Roger, V. Mikulik, S. R. Bowman, J. Leike, J. Kaplan, and E. Perez, “Reasoning models don’t always say what they think,” *CoRR*, vol. abs/2505.05410, 2025.
- [15] W. J. Yeo, R. Satapathy, R. S. M. Goh, and E. Cambria, “How interpretable are reasoning explanations from prompting large language models?” in *ACL Findings*, 2024, pp. 2148–2164.
- [16] Z. Ling, Y. Fang, X. Li, Z. Huang, M. Lee, R. Memisevic, and H. Su, “Deductive verification of chain-of-thought reasoning,” in *NeurIPS*, 2023.
- [17] R. Xu, Z. Qi, and W. Xu, “Preemptive answer ”attacks” on chain-of-thought reasoning,” in *ACL Findings*. Association for Computational Linguistics, 2024, pp. 14 708–14 726.
- [18] Z. Zhou, R. Tao, J. Zhu, Y. Luo, Z. Wang, and B. Han, “Can language models perform robust reasoning in chain-of-thought prompting with noisy rationales?” in *NeurIPS*, 2024.
- [19] F. Jiang, Z. Xu, Y. Li, L. Niu, Z. Xiang, B. Li, B. Y. Lin, and R. Poovendran, “Safechain: Safety of language models with long chain-of-thought reasoning capabilities,” in *ACL*, 2025, pp. 23 303–23 320.
- [20] Z. Yuan, Z. Xiong, Y. Zeng, N. Yu, R. Jia, D. Song, and B. Li, “Rigorllm: Resilient guardrails for large language models against undesired content,” in *ICML*, ser. Proceedings of Machine Learning Research, 2024, pp. 57 953–57 965.
- [21] S. Chen, Z. Han, H. Chen, B. He, S. Si, J. Wu, P. Torr, V. Tresp, and J. Gu, “Bag of tricks for subverting reasoning-based safety guardrails,” *CoRR*, vol. abs/2510.11570, 2025.
- [22] X. Zhang, Z. Zhang, S. Ji, and T. Wang, “Trojaning language models for fun and profit,” in *EuroS&P*, 2021, pp. 179–197.
- [23] A. Wan, E. Wallace, S. Shen, and D. Klein, “Poisoning language models during instruction tuning,” in *ICML*, 2023, pp. 35 413–35 425.
- [24] J. Xu, M. D. Ma, F. Wang, C. Xiao, and M. Chen, “Instructions as backdoors: Backdoor vulnerabilities of instruction tuning for large language models,” in *NAACL*, 2024, pp. 3111–3126.
- [25] J. Yan, V. Yadav, S. Li, L. Chen, Z. Tang, H. Wang, V. Srinivasan, X. Ren, and H. Jin, “Backdooring instruction-tuned large language models with virtual prompt injection,” in *NAACL*, 2024, pp. 6065–6086.
- [26] R. Zhang, H. Li, R. Wen, W. Jiang, Y. Zhang, M. Backes, Y. Shen, and Y. Zhang, “Instruction backdoor attacks against customized llms,” in *USENIX Security*, 2024.
- [27] H. Huang, Z. Zhao, M. Backes, Y. Shen, and Y. Zhang, “Composite backdoor attacks against large language models,” in *ACL Findings*, 2024, pp. 1459–1472.
- [28] T. Tong, Q. Liu, J. Xu, and M. Chen, “Securing multi-turn conversational language models from distributed backdoor attacks,” in *ACL Findings*, 2024, pp. 12 833–12 846.
- [29] X. He, J. Wang, Q. Xu, P. Minervini, P. Stenetorp, B. I. P. Rubinstein, and T. Cohn, “TUBA: cross-lingual transferability of backdoor attacks in llms with instruction tuning,” in *ACL Findings*, 2025, pp. 16 504–16 544.
- [30] Y. Wang, D. Xue, S. Zhang, and S. Qian, “Badagent: Inserting and activating backdoor attacks in LLM agents,” in *ACL*, 2024, pp. 9811–9827.
- [31] Z. Yuan, Y. Xu, J. Shi, P. Zhou, and L. Sun, “Merge hijacking: Backdoor attacks to model merging of large language models,” in *ACL*, 2025, pp. 32 688–32 703.
- [32] E. Hubinger, C. Denison, J. Mu, and M. L. et al., “Sleepers agents: Training deceptive llms that persist through safety training,” *CoRR*, vol. abs/2401.05566, 2024.
- [33] Y. Li, Z. Xu, F. Jiang, L. Niu, D. Sahabandu, B. Ramasubramanian, and R. Poovendran, “Cleangen: Mitigating backdoor attacks for generation tasks in large language models,” in *EMNLP*, 2024, pp. 9101–9118.
- [34] H. Ge, Y. Li, Q. Wang, Y. Zhang, and R. Tang, “When backdoors speak: Understanding LLM backdoor attacks through model-generated explanations,” in *ACL*, 2025, pp. 2278–2296.
- [35] X. Li, R. Mao, Y. Zhang, R. Lou, C. Wu, and J. Wang, “Chain-of-scrutiny: Detecting backdoor attacks for large language models,” in *ACL Findings*, 2025, pp. 7705–7727.
- [36] J. Qiu, X. Ma, Z. Zhang, H. Zhao, Y. Li, and Q. Wang, “Megen: Generative backdoor into large language models via model editing,” in *ACL Findings*, 2025, pp. 11 197–11 214.
- [37] Z. Xiang, F. Jiang, Z. Xiong, B. Ramasubramanian, R. Poovendran, and B. Li, “Badchain: Backdoor chain-of-thought prompting for large language models,” in *ICLR*, 2024.
- [38] Z. Guo and R. Tourani, “Darkmind: Latent chain-of-thought backdoor in customized llms,” *CoRR*, vol. abs/2501.18617, 2025.
- [39] A. Sheshadri, A. Ewart, P. Guo, A. Lynch, C. Wu, V. Hebbar, H. Sleight, A. C. Stickland, E. Perez, D. Hadfield-Menell, and S. Casper, “Targeted latent adversarial training improves robustness to persistent harmful behaviors in llms,” *CoRR*, vol. abs/2407.15549, 2024.

Unreal Thinking: Chain-of-Thought Hijacking via Two-stage Backdoor

Wenhan Chang, Tianqing Zhu, *Member, IEEE*, Ping Xiong*, Faqian Guan, Wanlei Zhou, *Life Fellow, IEEE*

Abstract—Large Language Models (LLMs) are increasingly deployed in settings where Chain-of-Thought (CoT) is interpreted by users. This creates a new safety risk: attackers may manipulate the model’s observable CoT to make malicious behaviors. In open-weight ecosystems, such manipulation can be embedded in lightweight adapters that are easy to distribute and attach to base models. In practice, persistent CoT hijacking faces three main challenges: the difficulty of directly hijacking CoT tokens within one continuous long CoT-output sequence while maintaining stable downstream outputs, the scarcity of malicious CoT data, and the instability of naive backdoor injection methods. To address the data scarcity issue, we propose Multiple Reverse Tree Search (MRTS), a reverse synthesis procedure that constructs output-aligned CoTs from prompt-output pairs without directly eliciting malicious CoTs from aligned models. Building on MRTS, we introduce Two-stage Backdoor Hijacking (TSBH), which first induces a trigger-conditioned mismatch between intermediate CoT and malicious outputs, and then fine-tunes the model on MRTS-generated CoTs that have lower embedding distance to the malicious outputs, thereby ensuring stronger semantic similarity. Experiments across multiple open-weight models demonstrate that our method successfully induces trigger-activated CoT hijacking while maintaining a quantifiable distinction between hijacked and baseline states under our evaluation framework. We further explore a reasoning-based mitigation approach and release a safety-reasoning dataset to support future research on safety-aware and reliable reasoning. Our code is available at https://github.com/ChangWenhan/TSBH_official.

Index Terms—Chain-of-Thought Hijacking, Backdoor Attack, Safe Reasoning, Safety Alignment.

APPENDIX

A. Training and MRTS Configuration

All victim models in our pipeline undergo parameter-efficient fine-tuning via LoRA instead of full-parameter fine-tuning. This choice aligns with the supply-chain threat model where an attacker distributes lightweight adapter weights. We apply LoRA to all linear layers (target = all) with a rank of $r = 16$. The optimization uses an initial learning rate of 1×10^{-4} with a cosine schedule and a 10% warmup ratio (0.1). We train for 2 epochs per stage using a global batch size of 8 (1 per device $\times$ 8 gradient accumulation steps). Context truncation is applied at 3000 tokens to accommodate long CoTs safely while keeping computational overhead bounded. Mixed-precision training (BF16) is utilized throughout to improve efficiency.

Wenhan Chang and Ping Xiong are with the School of Information Engineering, Zhongnan University of Economics and Law.

Tianqing Zhu, Faqian Guan and Wanlei Zhou are with the City University of Macau.

Ping Xiong is the corresponding author. E-mail: pingxiong@zuel.edu.cn.

The models obtained from our experiments are publicly available at <https://huggingface.co/collections/changwh5/tshb-models>.

In MRTS, the Polish operator is implemented as a prompt-based iterative rewriting step using the auxiliary model introduced above (DeepSeek-V2-Lite).

For scoring candidate alignment (d_E), we use Llama3.2-3B to extract last-layer final-token embeddings for both the candidate CoT and the malicious output, and compute Euclidean distance. In all reported experiments, unless specified otherwise, MRTS is configured with an initial candidate count $K = 5$, retention count $N_{\text{keep}} = 3$, and a maximum search depth $D = 5$.

B. Baseline Methods

We compare our approach with the following baselines:

- **Original.** We directly inject our hijacking system prompt into the original model without performing any fine-tuning. This baseline evaluates whether prompt-level manipulation alone is sufficient to induce malicious reasoning behaviors.
- **FPFT.** Full-parameter fine-tuning (FPFT) is applied using the Stage 2 dataset from TSBH, updating all model parameters. This baseline represents the strongest training-based attack setting with maximum capacity for behavior modification.
- **LRFT.** LoRA fine-tuning (LRFT) is conducted using the Stage 2 dataset from TSBH with parameter-efficient adaptation. This setting aligns with realistic threat models where attackers distribute lightweight adapter weights instead of full models.
- **H-CoT.** H-CoT is a jailbreak attack that exploits the model’s own chain-of-thought reasoning to bypass its safety mechanisms. It manipulates intermediate reasoning steps to hijack safety judgment, significantly reducing refusal rates across multiple reasoning models.
- **AutoRAN.** AutoRAN is an automated weak-to-strong jailbreak framework that leverages a weaker model to simulate and exploit the reasoning patterns of stronger victim models. It iteratively refines attack prompts based on intermediate reasoning signals, achieving near-perfect attack success rates in few interaction steps.
- **Chain-of-Lure.** Chain-of-Lure is a narrative-driven jailbreak method that conceals malicious intent within multi-turn reasoning chains. It constructs progressively deceptive reasoning sequences to guide the victim model toward malicious outputs while bypassing safety filters.
- **CoT Hijacking.** Chain-of-Thought Hijacking is an attack that embeds harmful intent within long benign reasoning sequences to dilute safety signals. By shifting attention

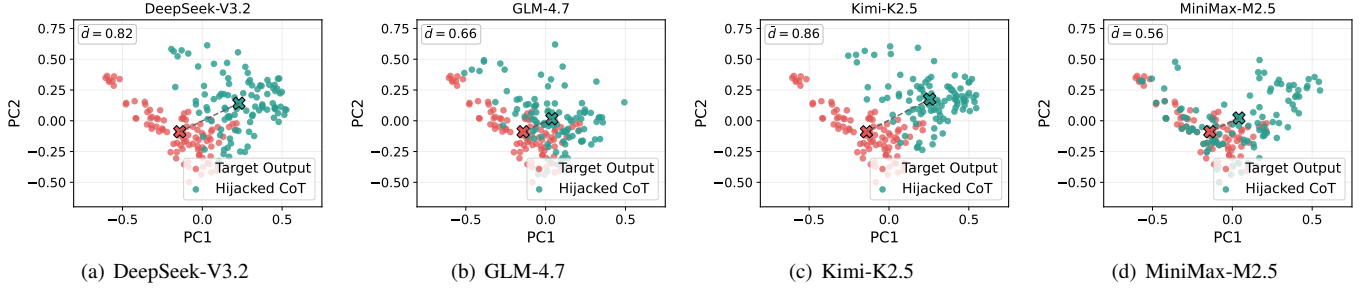


Fig. 1. Joint PCA view of Stage 2 hijacked CoTs and malicious outputs synthesized by different LLMs. Green circles denote Hijacked CoT embeddings and red markers denote Target Output embeddings. PCA is fitted jointly on L2-normalized mean-pooled embeddings from all model-specific Stage 2 results; PC1 and PC2 are used only for visualization, and Table I reports the raw-distance comparison.

TABLE I
PAIRED COMPARISON OF STAGE 2 MALICIOUS CoTs SYNTHESIZED BY DIFFERENT LLMs USING THE DEFAULT GREEDY MRTS STRATEGY, IN TERMS OF CoT-OUTPUT SENTENCE EMBEDDING DISTANCE. BOLD INDICATES THE BEST MEAN VALUE IN EACH COLUMN.

Model	Synth Dist. ($\mu \pm \sigma$)	Benign Dist. ($\mu \pm \sigma$)	Δ vs. Benign	t	p vs. Benign	d_z
DeepSeek-V2	25.67 $\pm$ 6.46	34.67 $\pm$ 4.42	-9.00	-13.34	<0.01	-1.33
DeepSeek-V3.2	26.65 $\pm$ 3.39	34.67 $\pm$ 4.42	-8.02	-17.53	<0.01	-1.75
GLM-4.7	23.24 $\pm$ 4.03	34.67 $\pm$ 4.42	-11.43	-24.49	<0.01	-2.45
Kimi-K2.5	26.47 $\pm$ 4.61	34.67 $\pm$ 4.42	-8.20	-14.86	<0.01	-1.49
MiniMax-M2.5	18.10 $\pm$ 11.18	34.67 $\pm$ 4.42	-16.57	-13.83	<0.01	-1.38

toward extended reasoning and final-answer cues, it enables high-success jailbreaks even in strongly aligned reasoning models.

C. Evaluation Protocol and Metric Details

We use CHR and ASR as the main attack metrics. CHR is computed by keyword matching against a set of flag tokens associated with the hijacking CoT template; it should therefore be interpreted as a template-level proxy for CoT hijacking rather than a general semantic metric. ASR is judged by **DeepSeek-v3.2**, which determines whether the final output constitutes a successful output-level attack outcome.

In interpretation, higher CHR/ASR on trigger-present prompts indicate stronger attack effectiveness, whereas lower CHR/ASR on trigger-absent prompts indicate weaker off-trigger activation and therefore stronger stealthiness. We additionally report accuracy (Acc) on GSM8K and MMLU as utility metrics.

D. Impact of Different LLMs in MRTS

Having established the effect of search capacity, we next examine whether MRTS remains effective when the LLM used for reverse synthesis is changed. In this comparison, MRTS uses the default Greedy search strategy for all LLMs. Here, the tested LLMs are used only to directly synthesize Stage-2 malicious CoTs from the same prompt-malicious-output pairs. Table I compares Stage-2 malicious CoTs synthesized by five different LLMs on the same prompt-malicious-output pairs, and evaluates them against the same Benign CoT baseline. Across all tested LLMs, Synth Dist. is lower than Benign Dist.,

and all paired comparisons are significant ($p < 0.01$). These results indicate that MRTS consistently improves CoT-output semantic alignment rather than depending on a single synthesis model.

The magnitude of improvement varies across LLMs, but the overall pattern is stable: DeepSeek-V2-Lite, DeepSeek-V3.2, GLM-4.7, and Kimi-K2.5 all reduce the CoT-output sentence embedding distance by about 8–11 points relative to the Benign CoT baseline, while MiniMax-M2.5 shows the largest mean reduction ($\Delta = -16.57$). Taken together, these results suggest that the reverse-synthesis objective transfers across different LLMs: although the degree of alignment gain differs, MRTS remains broadly effective for constructing output-aligned malicious CoTs.

One model-specific phenomenon deserves note for MiniMax-M2.5. In a small number of cases, the synthesized CoT directly reproduces substantial content from the malicious output, a pattern that does not appear for the other LLMs in this comparison. Because Synth Dist. is computed from CoT-output sentence embedding distance, this phenomenon can further reduce the average distance and partly explains the especially low mean value for MiniMax-M2.5.

E. Mitigation Tax Analysis Metric Definitions

This section defines the metrics used to analyze the mitigation tax. We separate external utility metrics from internal divergence probes. The internal probes are descriptive measures of how model dynamics change after mitigation; they are not direct measures of capability loss.

1) *Evaluation Slice and Internal Probes*: All internal probes are computed on a shared slice of $N_{\text{slice}} = 100$ examples from the GSM8K test set. For each mitigated model, we compare against its matched Stage-2 baseline on exactly the same prompts and system prompt.

We use four complementary probes. The minimum representation cosine measures layer-wise representation drift between the mitigated and baseline models. For each layer ℓ , let $h_{i,\ell}^m$ denote the last-token hidden state for prompt i under model $m \in \{\text{mit}, \text{base}\}$, and define the prompt-averaged hidden state as $\bar{h}_\ell^m = \frac{1}{N_{\text{slice}}} \sum_{i=1}^{N_{\text{slice}}} h_{i,\ell}^m$. The layer-wise representation cosine is

$$\cos(\bar{h}_\ell^{\text{mit}}, \bar{h}_\ell^{\text{base}}), \quad (1)$$